

Comenzado el	domingo, 12 de enero de 2025, 21:09
Estado	Finalizado
Finalizado en	domingo, 12 de enero de 2025, 21:35
Tiempo empleado	26 minutos 22 segundos
Puntos	9,00/25,00
Calificación	3,60 de 10,00 (36%)

Pregunta 1

Sin contestar

Se puntúa como 0 sobre 1,00

Al finalizar su ejecución un algoritmo de ramificación y poda la cola de prioridad:

Seleccione una:

- ☐ a. siempre queda vacía.
- ☐ b. puede contener nodos no factibles siempre que sean prometedores.
- ☐ c. puede contener nodos prometedores y no prometedores.
- ☐ d. solo puede contener nodos no prometedores.

El algoritmo de ramificación y poda puede detenerse o bien porque la cola de prioridad queda vacía o bien porque la prioridad del más prioritario de la cola es peor (menor o mayor dependiendo de si es un problema de maximización o minimización) que el valor de la mejor solución encontrada hasta el momento. Por tanto, no siempre queda vacía y en caso de que no lo esté, todos los nodos que permanecen en ella son no prometedores. Por otra parte, en la cola de prioridad nunca entran nodos no factibles. Así que la respuesta correcta es que solo puede contener nodos no prometedores.

La respuesta correcta es: solo puede contener nodos no prometedores.

Pregunta 2

Incorrecta

Se puntúa -0,33 sobre 1,00

¿Cuál es el coste de añadir una arista en un grafo dirigido con V vértices y A aristas, representado mediante *listas de adyacentes*?

Seleccione una:

- ☒ a. $O(V + A)$ ✖
- ☐ b. $O(V)$
- ☐ c. $O(V * A)$
- ☐ d. $O(1)$

Para añadir la arista $u \rightarrow v$ basta con añadir v a la lista de adyacentes de u , por ejemplo al principio o al final de la lista, lo que puede hacerse en tiempo constante.

La respuesta correcta es: $O(1)$

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

En el algoritmo de programación dinámica que resuelve el problema del producto encadenado de matrices, podemos rellenar la matriz:

Seleccione una:

- ☐ a. de arriba abajo y de derecha a izquierda.
- ☐ b. de arriba abajo y de izquierda a derecha.
- ☐ c. por diagonales comenzando en la diagonal del elemento del extremo superior derecho.
- ☒ d. por diagonales comenzando en la diagonal encima de la principal. ✓

Puesto que cada posición (i, j) depende de las posiciones (i, k) y $(k + 1, j)$ tales que $i \leq k \leq j - 1$, las cuales están respectivamente en su misma fila a la izquierda hasta la diagonal principal, y en la misma columna pero en filas inferiores hasta la diagonal principal, una posible forma de rellenar la matriz es por diagonales comenzando en la diagonal encima de la principal. El resto de opciones no garantiza que estén rellenas las posiciones necesarias. Sí podría rellenarse, sin embargo, de abajo arriba y de izquierda a derecha.

La respuesta correcta es: por diagonales comenzando en la diagonal encima de la principal.

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

En el algoritmo de Floyd para calcular los caminos mínimos entre cada par de vértices la matriz se puede rellenar en cada iteración:

Seleccione una:

- ☐ a. solamente de arriba abajo y de izquierda a derecha.
- ☐ b. solamente de arriba abajo y de derecha a izquierda.
- ☐ c. solamente por diagonales paralelas a la principal comenzando en la diagonal encima de la principal.
- ☒ d. de cualquier forma. ✓

En la iteración k -ésima cada posición (i, j) de la matriz necesita los valores (i, k) , (k, j) y el propio (i, j) de la iteración anterior. Pero se puede demostrar que en la iteración k -ésima los valores de la fila k y de la columna k no se modifican, por lo que la matriz se puede rellenar de cualquier forma sin que se pierdan los valores que se necesitan en cada momento.

La respuesta correcta es: de cualquier forma.

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

La fase de reconstrucción de todos los caminos mínimos entre todos los pares de vértices de un grafo en el algoritmo de Floyd tiene coste en el caso peor en $O(A)$ siendo A el número de aristas del grafo.

Seleccione una:

- ☐ a. Verdadero
- ☒ b. Falso ✓

Falso. La reconstrucción de todos los caminos tiene coste en $O(V^3)$ siendo V el número de vértices. Hay V^2 parejas de vértices cuyos caminos queremos reconstruir y la reconstrucción de cada uno de ellos está en $O(V)$.

La respuesta correcta es: Falso

Pregunta 6

Sin contestar

Se puntúa como 0 sobre 1,00

Dado un grafo dirigido implementado con *listas de adyacencia*, ¿cuál sería el coste de un algoritmo eficiente que busque un vértice con grado de entrada máxima, es decir, con grado de entrada $V - 1$?

Seleccione una:

- ☐ a. $O(A + V)$
- ☐ b. $O(A)$
- ☐ c. $O(1)$
- ☐ d. Ninguna de las anteriores.

- a. Cierto. Para encontrar un nodo con grado de entrada $V - 1$, tenemos que recorreremos todas las listas de adyacencia de cada vértice para ver si algún vértice aparece $V - 1$ veces. Por lo tanto el coste es $O(A + V)$
- b. Falso. Incorrecta. Es $O(A + V)$.
- c. Falso. Incorrecta. Es $O(A + V)$.
- d. Falso. La respuesta correcta es: $O(A + V)$

La respuesta correcta es: $O(A + V)$

Pregunta 7

Sin contestar

Se puntúa como 0 sobre 1,00

Supongamos una función recursiva que está definida de la siguiente manera:

$$f(1) = e_1$$

$$f(i) = \max(e_i, f(i-1) + e_i) \text{ para } i > 1$$

siendo $e_1, \dots, e_n$ expresiones numéricas y que queremos implementar un algoritmo de programación dinámica ascendente que calcule $\max_{i=1}^n f(i)$. ¿Qué cantidad de memoria adicional óptima necesita el algoritmo?

Seleccione una:

- ☐ a. $\theta(n)$
- ☐ b. $\theta(1)$
- ☐ c. $\theta(n^2)$
- ☐ d. $\theta(n \log n)$

Basta con utilizar una variable para guardar el valor de $f(i)$ que vamos calculando (desde 1 hasta n por ser ascendente), y otra para guardar el máximo de los valores de f calculados hasta el momento. Luego la respuesta correcta es $\theta(1)$.

La respuesta correcta es: $\theta(1)$

Pregunta 8

Incorrecta

Se puntúa -0,33 sobre 1,00

En un grafo no dirigido, para encontrar el camino más corto (con menos aristas) entre dos vértices se utiliza

Seleccione una:

- ☐ a. búsqueda en profundidad.
- ☐ b. búsqueda en anchura.
- ☒ c. ni búsqueda en profundidad ni en anchura. ✖
- ☐ d. tanto búsqueda en profundidad como en anchura.

La búsqueda en anchura, al ir recorriendo los vértices por distancias crecientes desde el origen, garantiza que encuentra el camino con menos aristas.

La respuesta correcta es: búsqueda en anchura.

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Indica la fracción (no nula) del último objeto elegido por el algoritmo voraz que resuelve el problema de la mochila real en el caso en que el peso límite es 25 y los objetos tienen los siguientes pesos y valores:

Objetos	Valores	Pesos
1	30	5
2	50	10
3	60	15

Seleccione una:

- ☒ a. $2/3$ ✓
- ☐ b. $1/3$
- ☐ c. 1
- ☐ d. $4/3$

Aplicando la estrategia voraz, se consideran por orden los objetos 1, 2 y por último el 3. Los objetos 1 y 2 caben completos y del objeto 3 solamente cabe $2/3$.

La respuesta correcta es: $2/3$

Pregunta 10

Sin contestar

Se puntúa como 0 sobre 1,00

Supongamos un sistema monetario con cantidad ilimitada de monedas de valores {1, 3, 6}. Se desea pagar la cantidad 10 con el menor número de monedas. Escribe separados por un espacio los valores de la última fila de la matriz rellenada por el algoritmo de programación dinámica que resuelve el problema.

Respuesta: ✗

Las filas se rellenan de arriba abajo y de izquierda a derecha obteniendo la siguiente tabla:

	0	1	2	3	4	5	6	7	8	9	10
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	1	2	3	4	5	6	7	8	9	10
2	0	1	2	1	2	3	2	3	4	3	4
3	0	1	2	1	2	3	1	2	3	2	3

Por tanto la solución es: 0 1 2 1 2 3 1 2 3 2 3

La respuesta correcta es: 0 1 2 1 2 3 1 2 3 2 3

Pregunta 11

Correcta

Se puntúa 1,00 sobre 1,00

El problema de la multiplicación encadenada de n matrices de forma óptima puede resolverse con una complejidad en tiempo en:

Seleccione una:

- ☒ a. $O(n^3)$ ✓
- ☐ b. $O(m^2)$ siendo m el número de filas y columnas de las n matrices
- ☐ c. $O(n^2)$
- ☐ d. $O(m^3)$ siendo m el número de filas y columnas de las n matrices

El algoritmo calcula para cada pareja de índices i, j tales que $i \leq j$ el número mínimo de multiplicaciones básicas para multiplicar las matrices de la i a la j . El cálculo de cada valor es lineal en n porque requiere considerar las distintas formas de colocar los paréntesis externos para realizar la última multiplicación. Puesto que la cantidad de parejas a calcular es del $O(n^2)$, el coste está en $O(n^3)$.

La respuesta correcta es: $O(n^3)$

Pregunta 12

Correcta

Se puntúa 1,00 sobre 1,00

Tenemos una pared en la que queremos colocar el mayor número de rollos con papel pintado sin que se solapen. Los tramos de papel pintado los colocaremos en vertical.

Considerando que la longitud de cada rollo de papel es la misma (y la necesaria para cubrir completamente de arriba a abajo la altura de la pared), pero su anchura no. ¿Cuál de las siguientes estrategias voraces es correcta para resolver el problema anterior?

Seleccione una:

- ☐ a. Elegimos cada vez un rollo cualquiera de los que tenemos disponibles
- ☐ b. No existe una estrategia voraz correcta para este problema
- ☐ c. Ordenamos los rollos por orden creciente en su longitud y elegimos el primero libre que quepa
- ☒ d. Ninguna de las anteriores. ✓ Cierto. La respuesta correcta es: Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa

- a. Falso. Incorrecta. Si elegimos en cada ocasión un rollo aleatorio no está garantizado obtener el óptimo. Por ejemplo, en una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta podría coger únicamente el primero y la óptima los dos últimos.
- b. Falso. Incorrecta. La estrategia voraz en la que ordenamos crecientemente por anchura los rollos y cogemos el primero libre que quepa es correcta
- c. Falso. Incorrecta. La longitud de los rollos es la misma por lo que esa ordenación no nos ayuda. Por ejemplo, en una pared de 70cm y tres rollos de 70cm, 50cm y 20cm. La estrategia propuesta cogería sólo el primero y la óptima los dos últimos.
- d. Cierto. La respuesta correcta es: Ordenamos los rollos por orden creciente de anchura y cogemos el primero libre que quepa

La respuesta correcta es: Ninguna de las anteriores.

Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos un grafo no dirigido *completo* G con $V > 2$ vértices y todas las aristas con valor 1. ¿Cuál de las siguientes afirmaciones es cierta?

Seleccione una:

- ☐ a. G tiene múltiples ARMs con costes diferentes.
- ☐ b. G no tiene árbol de recubrimiento mínimo (ARM).
- ☒ c. G tiene múltiples ARMs, todos con coste $V - 1$.  Verdadero.
- ☐ d. G tiene un único ARM con coste $V - 1$.

- a. Falso. No tiene sentido que los costes sean diferentes, algunos no serían *de coste mínimo*.
- b. Falso. Al ser completo también es conexo, y todo grafo conexo tiene ARM.
- c. Verdadero.
- d. Falso. Como el valor de todas las aristas es el mismo, hay diferentes formas de conseguir el ARM.

La respuesta correcta es: G tiene múltiples ARMs, todos con coste $V - 1$.

Pregunta 14

Incorrecta

Se puntúa 0,00 sobre 1,00

A la hora de convertir un vector en un montículo como primera fase para ordenarlo de menor a mayor por el método heapsort, ¿cuál es el caso peor (el caso que provoca que haya que hacer más trabajo)?

Seleccione una:

- ☒ a. Cuando el vector inicial está ordenado de mayor a menor. 
- ☐ b. Cuando el vector inicial está ordenado de menor a mayor.
- ☐ c. Todos los casos provocan la misma cantidad de trabajo.
- ☐ d. No se puede saber a priori.

Para ordenar de menor a mayor el vector tiene que convertirse en un montículo de máximos. El caso peor se da cuando el vector está inicialmente ordenado de menor a mayor porque cada elemento (en la primera mitad del vector) es menor que todos los siguientes y tiene que ser hundido hasta una hoja.

La respuesta correcta es: Cuando el vector inicial está ordenado de menor a mayor.

Pregunta 15

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es el coste de añadir una arista en un grafo no dirigido con V vértices y A aristas, representado mediante *listas de adyacentes*?

Seleccione una:

- ☐ a. $O(V)$
- ☒ b. $O(1)$ ✓
- ☐ c. $O(V + A)$
- ☐ d. $O(V * A)$

Para añadir la arista $u - v$ basta con añadir v a la lista de adyacentes de u y u a la lista de adyacentes de v , lo cual se puede hacer en tiempo constante.

La respuesta correcta es: $O(1)$

Pregunta 16

Sin contestar

Se puntúa como 0 sobre 1,00

La función

```
void funcion(vector<int> & v) {  
    for (int i = v.size() - 1; i > 0; --i) {  
        flotar_max(v, v.size(), i);  
    }  
}
```

Seleccione una:

- ☐ a. convierte un montículo de máximos en uno de mínimos.
- ☐ b. mueve los elementos del vector, pero no lo convierte necesariamente en un montículo.
- ☐ c. siempre deja el vector sin modificar.
- ☐ d. convierte un vector cualquiera en un montículo.

Un vector puede convertirse en un montículo de dos maneras distintas:

- recorriendo los elementos de izquierda a derecha y *flotando* cada elemento entre los anteriores (ya procesados), o
- recorriendo la primera mitad de los elementos de derecha a izquierda, *hundiendo* cada elemento entre los siguientes, que ya forman montículos.

Aquí se han mezclado las dos ideas, lo que no garantiza que el resultado final sea un montículo, aunque haya elementos que cambien de lugar.

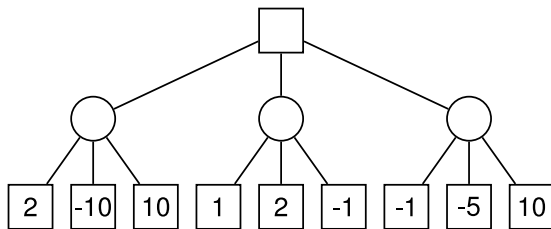
La respuesta correcta es: mueve los elementos del vector, pero no lo convierte necesariamente en un montículo.

Pregunta 17

Correcta

Se puntúa 1,00 sobre 1,00

Si utilizamos la poda alfa-beta en este árbol de juego, ¿cuántos nodos se podan?



Seleccione una:

- ☐ a. 1
- ☐ b. 3
- ☒ c. 2 ✓
- ☐ d. 0

En el primer subárbol inicialmente $\alpha = -\infty$ y $\beta = +\infty$ y mientras se calcula el mínimo de los hijos β va cambiando de valor a 2 y luego a -10 . No se produce ninguna poda y cuando termina de calcularse su valor, el nodo raíz toma valor $\alpha = -10$ y $\beta = +\infty$.

En el segundo subárbol tampoco se produce ninguna poda porque $\alpha = -10$ es menor que los valores que va tomando β que son primero 1 y después -1 . Cuando termina el segundo subárbol, el nodo raíz por tanto toma los valores $\alpha = -1$ y $\beta = +\infty$.

En el tercer subárbol, después del primer hijo β pasa a valer -1 que es igual a α y los dos últimos nodos no se visitan. Por tanto, la respuesta correcta es 2.

La respuesta correcta es: 2

Pregunta 18

Sin contestar

Se puntúa como 0 sobre 1,00

El coste de recorrer completamente un AVL con N nodos para producir una lista ordenada de sus elementos está en

Seleccione una:

- ☐ a. $O(N \log N)$
- ☐ b. $O(N)$
- ☐ c. $O(1)$
- ☐ d. $O(\log N)$

Al ser un árbol binario de búsqueda, para producir una lista ordenada basta con recorrer el árbol en inorden, lo que tiene un coste lineal respecto al número de nodos del árbol.

La respuesta correcta es: $O(N)$

Pregunta 19

Sin contestar

Se puntúa como 0 sobre 1,00

¿Cuál de las siguientes afirmaciones sobre el coste en tiempo de la estrategia voraz para resolver un problema en el que queremos maximizar el beneficio de n tareas de igual duración con plazo fijo, es correcta?

Seleccione una:

- ☐ a. $O(n)$
- ☐ b. $O(n \log n)$
- ☐ c. $O(1)$
- ☐ d. Ninguna de las anteriores.

- a. Falso. Incorrecta. No podemos asumir que las tareas están ordenadas de menor a mayor, por lo tanto el coste de ordenarlas será $O(n \log n)$. El resto de operaciones (incluyendo unir y buscar en conjuntos disjuntos) son, a efectos prácticos, constantes de ahí que no añadan más coste en tiempo.
- b. Cierto. No podemos asumir que las tareas vienen ordenadas por su beneficio, la mayor parte del coste en tiempo viene de ordenarlas de menor a mayor lo que supone un coste $O(n \log n)$, el resto de operaciones (incluyendo unir y buscar en conjuntos disjuntos) son, a efectos prácticos, constantes de ahí que no añadan más coste en tiempo.
- c. Falso. Incorrecta. No podemos asumir que las tareas están ordenadas de menor a mayor, por lo tanto el coste de ordenarlas será $O(n \log n)$. El resto de operaciones (incluyendo unir y buscar en conjuntos disjuntos) son, a efectos prácticos, constantes de ahí que no añadan más coste en tiempo.
- d. Falso. La respuesta correcta es: $O(n \log n)$

La respuesta correcta es: $O(n \log n)$ **Pregunta 20**

Sin contestar

Se puntúa como 0 sobre 1,00

Si el TAD de conjuntos disjuntos se implementa con la versión de *unión rápida*, ¿cuál es el coste de la operación **buscar** en el caso peor si el conjunto tiene N elementos?

Seleccione una:

- ☐ a. $O(N^2)$
- ☐ b. $O(1)$
- ☐ c. $O(N)$
- ☐ d. $O(\log N)$

En el caso peor la altura de un árbol que represente un conjunto puede ser lineal respecto al número de elementos de la partición.

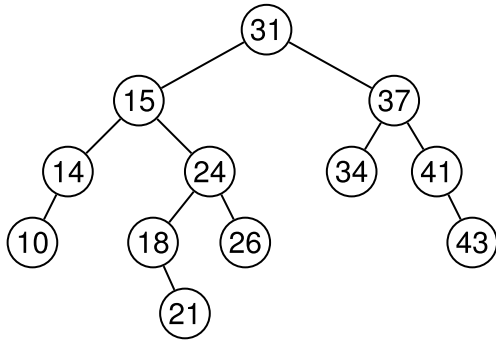
La respuesta correcta es: $O(N)$

Pregunta 21

Incorrecta

Se puntúa 0,00 sobre 1,00

Si en este árbol AVL eliminamos el valor 34, ¿cuántas rotaciones se producen (las rotaciones dobles se cuentan como una rotación)?



Seleccione una:

- ☐ a. 1
- ☒ b. 0 ✖
- ☐ c. 3
- ☐ d. 2

Tras eliminar el valor 34 el nodo con valor 37 pierde la condición de equilibrio, y hace falta una rotación simple a la izquierda para restablecerla. Eso hace que la raíz también se desequilibre, y haga falta una rotación doble izquierda-derecha para equilibrar el árbol.

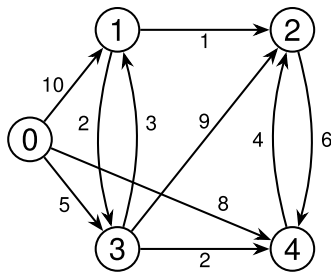
La respuesta correcta es: 2

Pregunta 22

Correcta

Se puntúa 1,00 sobre 1,00

Supón que ejecutamos el algoritmo de Dijkstra sobre este grafo utilizando como origen el vértice 0.



¿Cuántas veces cambia la prioridad del vértice 2 una vez insertado en la cola?

Seleccione una:

- ☐ a. 1
- ☐ b. 0
- ☒ c. 2 ✓
- ☐ d. 3

El vértice 2 se inserta en la cola cuando se relaja con las aristas que salen del vértice 3, con prioridad 14. Después cambia a 11 cuando se relaja con las aristas que salen del vértice 4. Y después vuelve a cambiar, a 9, cuando se relaja con las aristas que salen del vértice 1.

La respuesta correcta es: 2

Pregunta 23

Incorrecta

Se puntúa -0,33 sobre 1,00

En el algoritmo de Dijkstra con un grafo con V vértices y A aristas, ¿cuántas veces se invocan como mínimo las operaciones insertar y borrar el mínimo de la cola por cada vértice?

Seleccione una:

- ☒ a. V ✗
- ☐ b. 2
- ☐ c. 0
- ☐ d. 1

Los vértices no alcanzables desde el origen no se insertan nunca en la cola de prioridad.

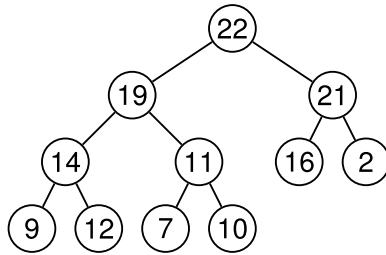
La respuesta correcta es: 0

Pregunta 24

Incorrecta

Se puntúa 0,00 sobre 1,00

En este montículo de máximos, ¿qué valores pueden haber sido el último en insertarse?



Seleccione una:

- ☐ a. Solamente el 10.
- ☐ b. Podría ser cualquiera.
- ☐ c. 11 y 10.
- ☒ d. 19, 11 y 10. ❌

El último elemento insertado se añadió en la hoja más a la derecha del último nivel y después fue flotado si hacía falta hacia la raíz. Puede haber sido el 10 y no haber necesitado ser flotado. Puede haber sido el 11 y haberse intercambiado con el 10, que ocupaba su posición. Pero no puede ser el 19, porque entonces el 11 ocuparía su posición antes de la última inserción, y el 11 no puede ser padre del 14 en un montículo de máximos. Por el mismo motivo no puede ser el 22 el último elemento insertado.

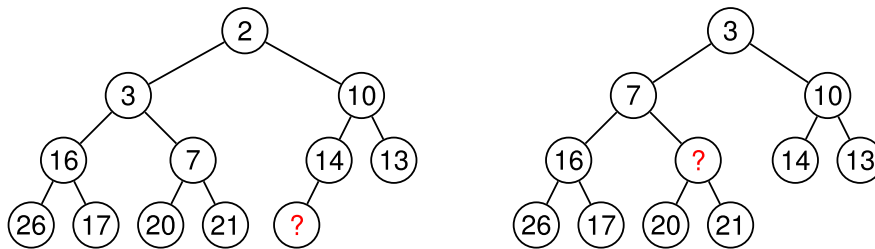
La respuesta correcta es: 11 y 10.

Pregunta 25

Incorrecta

Se puntúa 0,00 sobre 1,00

La eliminación del mínimo en el siguiente montículo de mínimos de la izquierda da como resultado el montículo de la derecha:



¿Cuál podría ser el valor oculto (marcado con una interrogación) si sabemos que todos los valores son distintos?

Seleccione una:

- ☐ a. 15, 18 o 19.
- ☒ b. Cualquier valor entre 8 y 19 (inclusive) que no esté ya. ✗ Falso. No puede ser el 9, por ejemplo, porque el 9 no puede ser hijo del 14.
- ☐ c. Cualquier valor mayor que 14 que no esté ya.
- ☐ d. Podría ser cualquier entero (no tenemos suficiente información) distinto de los que ya están.

Por ser hijo del 14 antes de la eliminación sabemos que tiene que ser mayor que 14. Por ser padre del 20, tiene que ser menor que 20. Como el 16 y el 17 ya están, solamente puede ser 15, 18 o 19.

- Cierto.
- Falso. No puede ser el 9, por ejemplo, porque el 9 no puede ser hijo del 14.
- Falso. No puede ser el 25, por ejemplo, porque el 25 no puede ser padre del 20 en un montículo de mínimos
- Falso. No puede ser el 1, por ejemplo, porque el 1 no puede ser hijo del 14.

La respuesta correcta es: 15, 18 o 19.